

DESCRIPTION D'UNE MISSION | BTS SIO

Prénom – Nom	RAPHEHISON Mandresy	
N° mission	F-06	
Option	☐ SISR	☒ SLAM
Situation	☒ Formation	☐ Entreprise
Lieu de réalisation	CFA Campus Montsouris, Paris	[Logo Campus Montsouris]
Période de réalisation	Du : 08/06/2025 Au : 08/06/2025	
Modalité de réalisation	☒ VÉCUE ☐ OBSERVÉE	
Intitulé de la mission	Introduction au framework Laravel : controller, vue Blade et formulaire	
Description du contexte	Dans le cadre du cours de développement web au CFA Campus Montsouris, j'ai réalisé une première prise en main du framework PHP Laravel. L'objectif était de comprendre l'architecture MVC de Laravel, le routage, la création d'un controller, l'utilisation du moteur de templates Blade et la gestion d'un formulaire.	
Ressources et outils utilisés	- VS Code - Laravel (PHP) - Artisan (CLI Laravel) - Composer (gestionnaire de dépendances PHP) - Serveur local PHP (php artisan serve) - Navigateur web pour les tests	
Résultat attendu	- Routes définies dans web.php pointant vers un controller - Un controller gérant l'affichage d'une vue et la réception d'un formulaire - Des vues Blade utilisant les directives (@extends, @section, @yield) - Un formulaire HTML soumis en POST avec protection CSRF (@csrf)	
Contraintes	- Respecter l'architecture MVC imposée par Laravel - Utiliser les conventions Laravel (nommage des controllers, des routes) - Utiliser les directives Blade plutôt que du PHP brut dans les vues	
Compétences associées (voir tableau)	- B2.1 – Concevoir et développer une solution applicative - B2.1 – Exploiter les ressources du cadre applicatif (framework)	

Description des étapes de réalisation (démarche, méthodes, techniques utilisées)

1. DÉCOUVERTE DE L'ARCHITECTURE LARAVEL

Laravel suit le patron MVC (Modèle - Vue - Contrôleur). Les fichiers sont organisés en dossiers distincts : routes/ pour le routage, app/Http/Controllers/ pour les controllers, et resources/views/ pour les vues Blade.
Le serveur de développement est lancé via la commande Artisan : php artisan serve.

2. DÉFINITION DES ROUTES (web.php)

Les routes sont déclarées dans routes/web.php. Une route GET affiche le formulaire et une route POST reçoit les données soumises :

```
Route::get('/formulaire', [MonController::class, 'index']);
Route::post('/formulaire', [MonController::class, 'store']);
```

3. CRÉATION DU CONTROLLER (Artisan)

Le controller a été généré via la commande Artisan :

```
php artisan make:controller MonController
```

Il contient deux méthodes :

- index() : retourne la vue Blade du formulaire via return view('formulaire')

- store(Request \$request) : récupère les données POST via \$request->input() et les passe à une vue de confirmation

4. VUES BLADE

Les vues utilisent le moteur de templates Blade avec ses directives :

- @extends('layout') pour hériter d'un layout commun
- @section('content') / @endsection pour définir le contenu de la section
- @yield('content') dans le layout pour injecter le contenu
- {{ \$variable }} pour afficher une variable (échappement automatique du HTML)

5. FORMULAIRE AVEC PROTECTION CSRF

Le formulaire HTML est inclus dans une vue Blade avec la directive @csrf qui génère automatiquement un token de sécurité pour protéger la soumission :

```
<form method="POST" action="/formulaire">
    @csrf
    <input type="text" name="nom">
    <button type="submit">Envoyer</button>
</form>
```

Sans @csrf, Laravel rejette automatiquement la requête POST (protection XSRF).

Conclusion	Cette introduction à Laravel m'a permis de comprendre la logique d'un framework MVC et ses avantages par rapport au PHP natif : séparation claire des couches, routage centralisé, protection CSRF intégrée et moteur de templates Blade. La prise en main d'Artisan comme outil de génération de code est également un acquis important pour gagner en productivité sur les projets Laravel.
Productions associées	- routes/web.php : définition des routes GET et POST - app/Http/Controllers/MonController.php : logique d'affichage et de traitement - resources/views/ : vues Blade (layout, formulaire, confirmation)